

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

Caching هو السر وراء تطبيقات أسرع وتجربة مستخدم سلسلة، حيث يقلل زمن الوصول للبيانات ويخفف الضغط على السيرفرات وقواعد البيانات علشان كذا ال **PFF** ده شامل ال **Cching**

Caching

يعني إيه **Caching** ؟

هو عملية تخزين البيانات اللي بنطلبها كتير أو اللي حسابها بياخد وقت طويل في مكان سريع جداً غالباً ال **(RAM)**، عشان لما نحتاجها تاني منروشح نجيبها من المصدر الأصلي ("البطيء") "زي الداتابيز أو **API** خارجي.

مثال من الحياة:

لو حد سألك: "عشرة في عشرة بكام؟" هترد فوراً "100". إنت محسبتهاش دلوقتي، إنت جبتها من ال **Cache** (ذاكرتك) لكن لو سألك "347 في 89"، هتطلع ورقة وقلم (الداتابيز) وتحسبها، وبعد ما تعرف الناتج هتخفظه في ذاكرتك عشان لو سألك تاني تجاوبه فوراً.

ليه بنستخدمه في البرمجة؟

1. السرعة: ال **(Performance)** ال **RAM** أسرع بآلاف المرات من الهارد ديسك **(Database)** .
2. تقليل الحمل: **(Reduce Load)** بدل ما الداتابيز يجيلها 1000 طلب في الثانية لنفس المعلومة، هيجيلها طلب واحد، وال 999 الباقيين يتخدموا من ال **Cache** .
3. توفير التكلفة: لو بتستخدم **API** خارجي بياخد فلوس على كل طلب، ال **Cache** هيوفرلك فلوس كتير.

في ال **Web** الدنيا بتمشي إزاي؟

لما تعمل **Request** <=== **GET /api/products**



لو مفيش **Cache**:

الـ **API** يروح على **Database** & ينفذ **Query** & يرجع **Result** & بيعتالك
لو عملت نفس الـ **Request** بعدها بثنائية هيعيد نفس الخطوات = وقت وضغط زيادة

لو فيه **Cache**:

أول مرة: يجيب من **Database** ويحط نسخة في **Cache**

المرات اللي بعدها: يجيب من **Cache** مباشرة

⚙️ أنواع الـ **Caching** في **ASP.NET Core**

1 Response Caching :: يقلل تنفيذ الكود عن طريق تخزين الـ **Response** فترة معينة بدل ما
يتبني من الصفر كل مرة

Response Caching = السيرفر ما بيخزنش الحاجة ❌

المتصفح أو الـ **Proxy** أو الـ **CDN** هو اللي بيخزنها ✅

• إعداد **Response Caching** في **ASP.NET Core**

إضافة الخدمة // `builder.Services.AddResponseCaching();`

• استخدامه في **Controller**

هنا:

• **Duration = 60** → السماح بتخزين الـ **Response** لمدة 60 ثانية

• **Location = Any** → مسموح يتخزن:

• في المتصفح (**Browser Cache**) أو في **Proxy** أو في **CDN**

```
[HttpGet]
[ResponseCache(Duration = 60, Location = ResponseCacheLocation.Any)]
public IActionResult GetTime()
{
    return Ok(new
    {
        ServerTime = DateTime.Now.ToString("T")
    });
}
```

4 إيه اللي بيحصل فعليًا؟

1. أول **Request** يجي:

• السيرفر ينفذ **GetTime** يرد بالوقت الحالي ويحط **Header** زي

Cache-Control: public,max-age=60

المتصفح (أو الـ **Proxy/CDN**) يشوف الهيدر، ويخزن الـ **Response**. أي **Request** ثاني لنفس الـ **URL** خلال 60 ثانية: المتصفح/الـ **Proxy** يدبك نفس النتيجة من الكاش مش لازم يرن الكود ثاني

* الكاش هنا عند العميل أو الشبكة، مش جوه السيرفر

إعدادات مهمة في **[ResponseCache]**

```
[ResponseCache(  
    Duration = 60,  
    Location = ResponseCacheLocation.Any,  
    NoStore = false)]
```

• **Duration** → مدة الكاش بالثواني

• **Location:**

• **Any** → مسموح يتخزن في أي مكان

• **Client** → في المتصفح بس

• **None** → ممنوع يتخزن

• **NoStore = true** → ممنوع تخزين خالص يعني **Disable** لأي **Cache**

2 **Output Caching**:: السيرفر نفسه بيخزن الـ **Response** الجاهز في الـ رام

يعني: مفيش **Database** & مفيش **Business Logic** مفيش **Controller Execution**

⚡ رد فوري

تفعيله في **Program.cs**

builder.Services.AddOutputCache(); // إضافة **Output Caching**



```

[HttpGet]
[OutputCache(Duration = 60)]
public IActionResult GetTime()
{
    return Ok(new
    {
        ServerTime = DateTime.Now.ToString("T")
    });
}

```

استخدام بسيط في Controller

هنا:

- أول **Request** : الكود يتنفذ النتيجة تتخزن في كاش السيرفر أي **Request** ثاني لنفس الـ **URL** خلال 60 ثانية : السيرفر يرد من الكاش مش بيدخل جوه **GetTime** أصلاً

Policies = قواعد تقول للسيرفر "إزاي تخزن الكاش؟"

بدل ما تقول : كاش 30 ثانية وخلص"

تقدر تقول: كاش 30 ثانية + لكل **pollId** نسخة مختلفة"

Policy + Vary = اعمل كاش منفصل لكل **pollId**

المشكلة عندك : Endpoint 

```

/api/polls/1/questions
/api/polls/2/questions

```

✗ من غير إعداد خاص السيرفر ممكن يعتبرهم نفس الطلب يعمل كاش واحد بس

الحل  : **SetVaryByRouteValue("pollId")**

دي بتقول للسيرفر: كل رقم **pollId** اعمله كاش لوحده"

مثال بسيط جداً بدل ما تحط كل الملفات في فولدر واحد 

تعمل : **Poll 1** & **Poll 2** & **Poll 3**  كل واحد له نسخه 

/polls/1/questions

اللي بيحصل فعلياً أول طلب

يدخل الداتا بيز يخزن نسخة باسم "1"

ثاني طلب لنفس الرقم **/polls/1/questions** يرجع من الكاش فوراً



/questions2/polls/

طلب لرقم مختلف

يعمل نسخة جديدة للرقم 2

الخلاصة النهائية كل = pollId كاش مستقل لوحده

```
builder.Services.AddOutputCache(options =>
{
    options.AddPolicy("PollQuestionsCache", policy =>
    {
        policy.Expire(TimeSpan.FromSeconds(30)) // الكاش يعيش 30 ثانية
        .SetVaryByRouteValue("pollId"); // pollId كاش منفصل لكل
    });
});

[HttpGet("{pollId}/questions")]
[OutputCache(PolicyName = "PollQuestionsCache")]
public IActionResult GetPollQuestions(int pollId)
{
    // Request لـ /api/polls/1/questions
    // هيكاش لوحده غير /api/polls/2/questions
}
```

```
builder.Services.AddOutputCache(options =>
{
    options.AddPolicy("PollQuestionsCache", X => x.Cache().Expire(TimeSpan.FromSeconds(30)))
})
```

اعمل كاش لأي Response يطبق عليه الـ Policy لمدة 30 ثانية بس كده

يعني ايه `x.Cache()`؟

دي بتقول: الـ Response ده ينفع يتخزن في الكاش "لأن Output Caching" عنده قواعد تمنع الكاش لبعض الحالات زي Authorization أو POST أحيانًا

فالسطر ده بيؤكد: اسمح بالكاش

3 يعني إيه Memory Cache؟

Memory Cache = تخزين البيانات في RAM بتاعة السيرفر

علشان لو نفس الداتا اتطلبت تاني: بدل ما نروح الداتا بيز كل مرة ناخذها من الكاش (الذاكرة) اللي هي أسرع بكثير يعني Memory Cache = مخزن مؤقت للداتا جوه السيرفر نفسه.

بدون كاش: User → Server → Database → Server → User

أول مرة

User → Server → Database → Cache → User Memory Cache مع

بعد كده

User → Server → Cache → User ⚡

إزاي أستخدم IMemoryCache في

ASP.NET Core؟



تسجيل **Memory Cache** في **Program.cs** (كثير من الـ **Templates** بتضيفها تلقائيًا، بس نوضح):

builder.Services.AddMemoryCache();

حقن **IMemoryCache** في **Service** أو **Controller**

مثال في **Service**:

```
public class QuestionService : IQuestionService
{
    private readonly ApplicationDbContext _context;
    private readonly IMemoryCache _cache;

    public QuestionService(ApplicationDbContext context, IMemoryCache cache)
    {
        _context = context;
        _cache = cache;
    }

    // ...
}
```

```
private const string _cachePrefix = "availableQuestion";

public async Task<Result<IEnumerable<QuestionResponse>>> GetAvailableAsync(int PollId,
    CancellationToken cancellationToken = default)
{
    var cacheKey = $"{_cachePrefix}-{pollId}";

    var questions = await _memoryCache.GetOrCreateAsync(cacheKey,
        cacheEntry =>
        {
            cacheEntry.SlidingExpiration = TimeSpan.FromMinutes(5);

            return _context.Questions
                .Where(x => x.PollId == pollId && x.IsActive)
                .Include(x => x.Answers)
                .Select(q => new QuestionResponse(
                    q.Id,
                    q.Content,
                    q.Answers
                        .Where(a => a.IsActive)
                        .Select(a => new Contracts.Answers.AnswerResponse(a.Id, a.Content))
                ))
                .AsNoTracking()
                .ToListAsync(cancellationToken);
        });
    return Result.Success<IEnumerable<QuestionResponse>>(questions!);
}
```

& الفكرة العامة للكود

*يجيب أسئلة **Poll**

* لو موجودة في الكاش → يرجعها فورًا ⚡

* لو مش موجودة → يجيبها من **DB** ويحفظها في الكاش

كل **Poll** له كاش خاص بيه **var cacheKey = \$"{_cachePrefix}-{pollId}"**



```
var questions = await _memoryCache.GetOrCreateAsync(
    cacheKey,
    cacheEntry =>
```

معناها لو الكاش فيه قيمة → رجّعها لو مفيش → نفّذ الكود جوه واحفظ النتيجة

cacheEntry.SlidingExpiration = TimeSpan.FromMinutes;(5)

Sliding Expiration يعني: الكاش يتجدد كل ما حد يستخدمه

مثال: حد طلب الداتا الساعة 1:00

تاني حد طلبها 1:04 العد يبدأ من 1:04 تاني

🔥 ممتاز للداتا اللي بتتطلب كثير

Memory Cache عيوب ❌

❌ يضع لو التطبيق **Restart**

❌ مش مشترك بين السيرفرات

عندك كذا سيرفر وعاليز كاش مشترك → ساعتها **Redis (Distributed Cache)**

❌ حجم الرام محدود

🌀 امتى تستخدمه؟

مثالي لـ: بيانات بتتطلب كثير & بيانات مش بتتغير بسرعة & تطبيق على سيرفر واحد

• زي: قائمة الدول & إعدادات الموقع & منتجات & **Categories**

Distributed Cache

◆ تعريف بسيط جدًا

Distributed Cache = كاش خارجي مش جوه السيرفر بدل ما البيانات تتخزن في **RAM** بتاعة

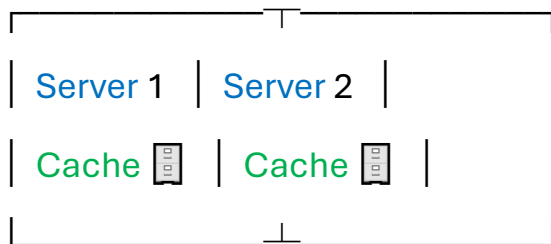
برنامجك بتتخزن في سيرفر تاني مخصص للكاش زي (Redis) بحيث كل السيرفرات تقدر توصله في نفس الوقت



🎯 له محتاجينه؟ تخيل إن مشروعك اشتغل على أكثر من سيرفر:

Load Balancer

↓



❌ المشكلة مع Memory Cache

المستخدم يعمل Login على Server 1 -> Token يتخزن في Memory Cache بتاع Server 1 ->

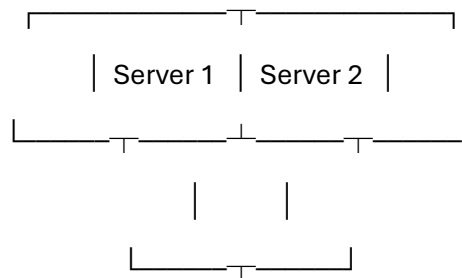
المستخدم يعمل Request ثاني ووصل لـ Server 2 -> Server 2 مش لاقى Token ❌

لأن Memory Cache منفصل لكل سيرفر

✅ الحل مع Distributed Cache

Load Balancer

↓



↓

Redis Cache

(مشترك بين الكل)



✓ كل السيرفرات بتتكلم مع نفس الكاش

✓ Memory Cache و Distributed Cache ثانياً: الفرق بين

المقارنة	Memory Cache	Distributed Cache
مكان الكاش	السيرفر RAM	(مثلاً Redis) خارج السيرفر
مشارك بين السيرفرات	لا ✗	آه ✓
السرعة	أسرع ⚡	أبطأ (Network)
لو السيرفر وقع	الكاش اتمسح ✗	الكاش موجود ✓
مناسب لـ	سيرفر واحد	أكثر من سيرفر
مثل	IMemoryCache	Redis / SQL Cache

✓ ثالثاً: أشهر Distributed Cache Implementations

1 Redis (الأشهر والأقوى) ::

Redis = In-Memory Database سريعة جداً مخصصة للـ Caching

2 SQL Server Distributed Cache ::

بيخزن الكاش في جدول في SQL Server أبطأ من Redis لكن مش محتاج Service جديد

3 Distributed Memory Cache (للتطوير بس)

بيحاكي Distributed Cache في RAM للتطوير والاختبار فقط مش للـ Production

✓ رابعاً IDistributedCache في ASP.NET Core :

ده Interface جاهز في ASP.NET Core بيديك Methods للتعامل مع الكاش:



سابعاً: استخدامها في QuestionService ✓

```
private readonly ICacheService _cacheService;
private readonly ILogger<QuestionServices> _logger;
private const string _cachePrefix = "availableQuestion";

public async Task<Result<IEnumerable<QuestionResponse>>> GetAvailableAsync(int PollId, string UserId,
    CancellationToken cancellationToken = default)
{
    // إنشاء مفتاح الكاش الفريد لكل Poll
    var cacheKey = $"{_cachePrefix}-{PollId}";

    // محاولة جلب البيانات من الكاش أولاً
    var cachedQuestion = await _cacheService.GetAsync<IEnumerable<QuestionResponse>>
        (cacheKey, cancellationToken);

    // تهيئة قائمة الأسئلة
    IEnumerable<QuestionResponse> Question = []

    if (cachedQuestion is null)
    {
        // لو مش موجودة في الكاش، جليها من قاعدة البيانات
        _logger.LogInformation("Select Questions From database");

        Question = await _context.Questions
            .Where(x => x.PollId == PollId && x.IsActive) // والنشطة فقط Poll الأسئلة الخاصة بالـ
            .Include(x => x.Answers) // جلب الإجابات المرتبطة بكل سؤال
            .Select(q => new QuestionResponse(
                q.Id,
                q.Content,
                q.Answers
                    .Where(a => a.IsActive) // الإجابات النشطة فقط
                    .Select(a => new Contracts.Answer.AnswerResponse(a.Id, a.Content))
            ))
            .AsNoTracking() // لتحسين الأداء عند القراءة فقط
            .ToListAsync(cancellationToken);
    }
    else
    {
        // إذا كانت موجودة في الكاش، استخدمها مباشرة
        _logger.LogInformation("Get Questions From cache");
        Question = cachedQuestion;
    }

    // تحديث الكاش بالقيم الجديدة أو المأخوذة من الداتا بيز
    await _cacheService.SetAsync(cacheKey, Question, cancellationToken);

    // إرجاع النتيجة بنجاح
    return Result.Success<IEnumerable<QuestionResponse>>(Question);
}
```

HybridCache

1 تعريف Hybrid Cache

Hybrid Cache هو مزيج بين:

- **Memory Cache (Local Memory Cache)**: تسريع جداً لأنه موجود في RAM الخاصة بالسيرفر.
- **Distributed Cache**: مشترك بين كل السيرفرات، زي Redis أو SQL Server Cache

فكرة **Hybrid Cache** : نوع جديد من الكاش ظهر في **.NET 9**.

أول حاجة يلاقي البيانات في **Local Memory Cache** → لو مش موجودة يروح يجيبها من **Distributed Cache** → لو مش موجودة هناك يجيبها من **Database**.

يعنى :: **Memory Cache** سريع جداً + **Distributed Cache** = **HybridCache**

🎯ليه جاء **HybridCache** أصلاً؟

المشكلة مع كل نوع لوحده

مشكلة **Memory Cache** :: سريع جداً مش مشترك بين السيرفرات لو السيرفر وقع الكاش اتمسح

مشكلة **Distributed Cache (Redis)** :: مشترك بين السيرفرات يصمد لو السيرفر وقع أبطأ لأن فيه **Network Call** ممكن يقع **Redis** نفسه

الحل **HybridCache** : سريع زي **Memory Cache** موزع زي **Redis**

لو **Redis** وقع → يرجع لـ **Memory Cache** لو السيرفر وقع **Redis** → موجود

رحلة الـ **Request** ::

Request يجي == > يبحث في **L1 (Memory)** أولاً == > لقي؟ → يرجع من **Memory** فوراً
مش لاقى؟ يبحث في **L2 (Redis)** لقي؟ → يحط في **L1** ويرجع مش لاقى؟ يروح الداتابيز يحط
النتيجة في **L1** و **L2** مع بعض يرجع للـ **Client**

الكاش انتهى == > جوا في نفس الوقت **HybridCache 1000 Request** بيخلي
Request واحد بس يروح **DB** الباقي 999 بيستنوا بهدوء النتيجة بتيجي وبتتحت في الكاش
الكل بياخد نفس النتيجة

إزاي أستخدم **HybridCache** ؟

Install-Package Microsoft.Extensions.Caching.Hybrid

builder.Services.AddHybridCache();

2 سجّله في **Program.cs**



استخدامه في Service

```
public async Task<Result<IEnumerable<QuestionResponse>>> GetAvailableAsync(int PollId, string UserId,
CancellationToken cancellationToken = default)
{
    // 3. تطبيق الـ Hybrid Cache
    var cacheKey = $"{_cachePrefix}-{PollId}";

    // بالضرورة factory داخل الـ Entry configuration مباشرة بدون الحاجة لـ GetOrCreateAsync يستخدم HybridCache ملاحظة: الـ
    var questions = await _hybridCache.GetOrCreateAsync<IEnumerable<QuestionResponse>>(
        cacheKey,
        async cacheEntry => await _context.questions
            .Where(x => x.PollId == PollId && x.IsActive)
            .Include(x => x.Answers)
            .Select(q => new QuestionResponse(
                q.Id,
                q.Content,
                q.Answers
                    .Where(a => a.IsActive)
                    .Select(a => new Contracts.Answer.AnswerResponse(a.Id, a.Content))
            ))
            .AsNoTracking()
            .ToListAsync(cancellationToken);

    return Result.Success<IEnumerable<QuestionResponse>>(questions!);
}
```

